

计算机网络

——2025-2026 第二学期

课程教师：周玉晨

联系方式：zhouyuchen@besti.edu.cn

时 间：2026年6月21日



知识点复习

随堂练习

- TCP协议的基本首部长度是()字节。

(在学习通上作答)

- A. 16
- B. 20
- C. 32
- D. 8



知识点复习

随堂练习

- 以下关于 TCP 与 UDP 协议特点的比较中,错误的是()。
(在学习通上作答)
 - A. TCP 提供可靠的报文传输,UDP 提供尽力而为的交付
 - B. TCP 基于字节流,UDP 基于报文
 - C. TCP 面向连接,UDP 无连接
 - D. TCP 传输速率高于 UDP



知识点复习

随堂练习

- 在停止等待协议中使用计时器的主要目的是()?
(在学习通上作答)
 - A. 计算已发数据的数目
 - B. 表示下一个要发送的数据序号
 - C. 超时重发
 - D. 分配序号



章节大纲

1	运输层协议概述
2	用户数据报协议UDP
3	传输控制协议TCP概述
4	可靠传输的工作原理
5	TCP报文段的首部格式
6	TCP可靠传输的实现
7	TCP 的流量控制
8	TCP的拥塞控制
9	TCP的运输连接管理



TCP可靠传输的实现

以字节为单位的滑动窗口

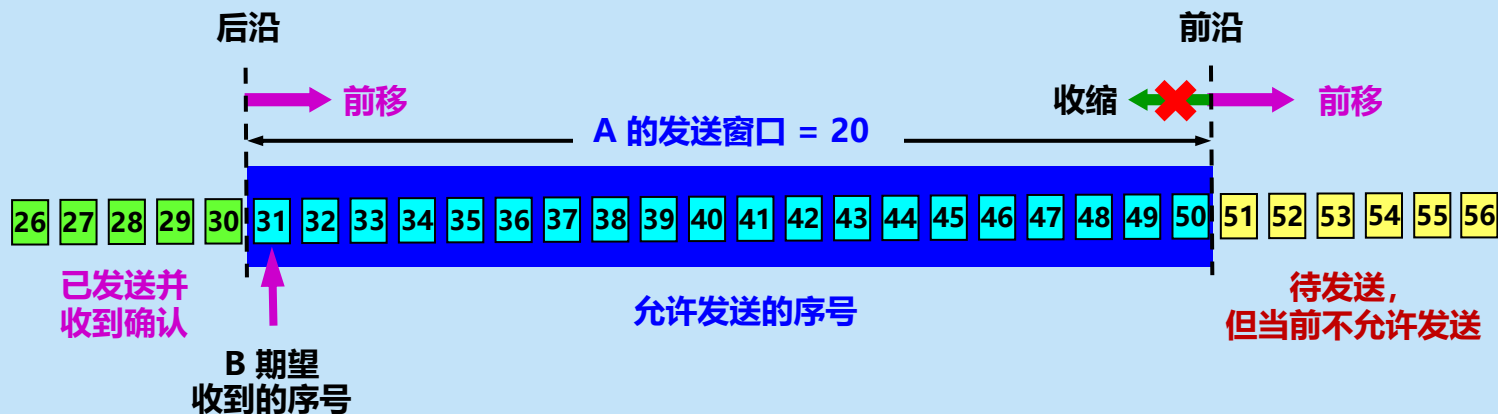
- TCP 使用**流水线传输**和**滑动窗口协议**实现高效、可靠的传输
- TCP 的滑动窗口是**以字节为单位的**
- 发送方 A 和接收方 B 分别维持一个**发送窗口**和一个**接收窗口**
- **发送窗口**：在没有收到确认的情况下，发送方可以**连续**把窗口内的数据**全部发送**出去。凡是已经发送过的数据，在未收到确认之前都必须**暂时保留**，以便在超时重传时使用
- **接收窗口**：只允许接收**落入**窗口内的数据



TCP可靠传输的实现

发送窗口

- A 根据 B 给出的窗口值，构造出自己的发送窗口。
- 发送窗口里面的**序号**表示允许发送的序号。
- 窗口越大，发送方就可以在收到对方确认之前连续发送更多的数据，因而可能获得更高的传输效率。

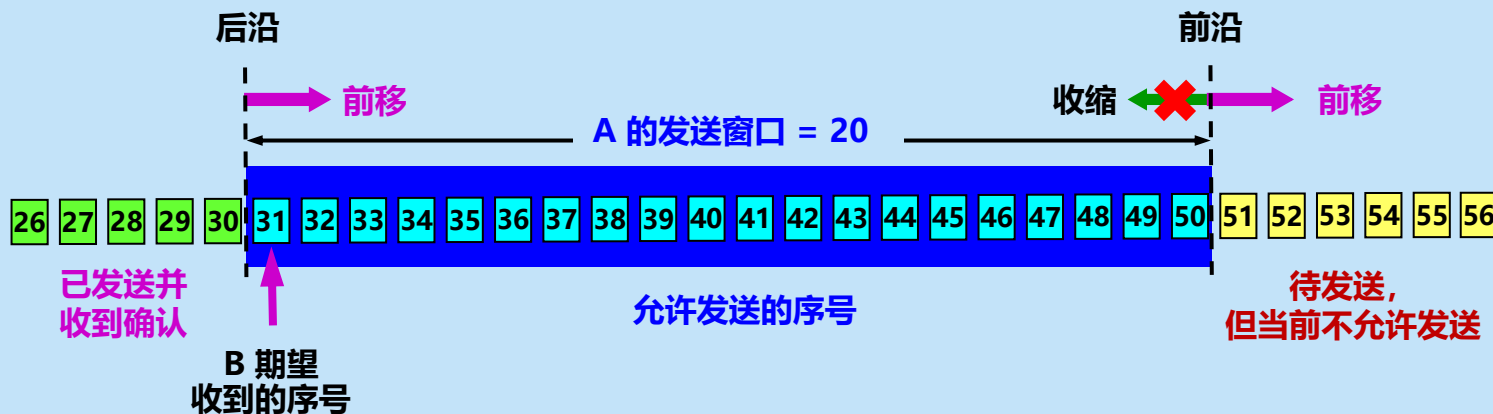




TCP可靠传输的实现

发送窗口

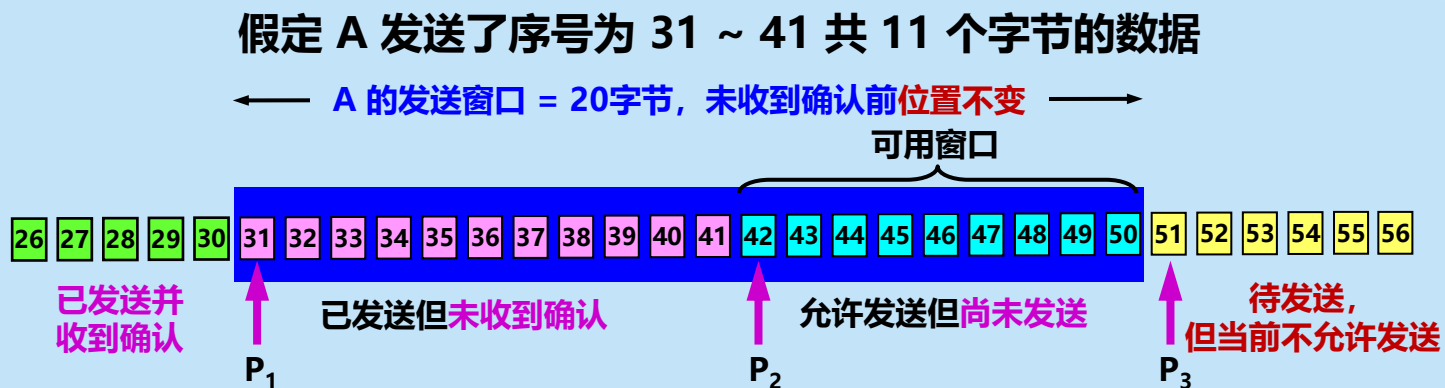
- A 根据 B 给出的窗口值，构造出自己的发送窗口。
- 发送窗口里面的**序号**表示允许发送的序号。
- 窗口越大，发送方就可以在收到对方确认之前连续发送更多的数据，因而可能获得更高的传输效率。





TCP可靠传输的实现

发送窗口



P_1 = 后沿, P_2 = 当前, P_3 = 前沿。

$P_3 - P_1$ = A 的发送窗口 (又称为通知窗口)

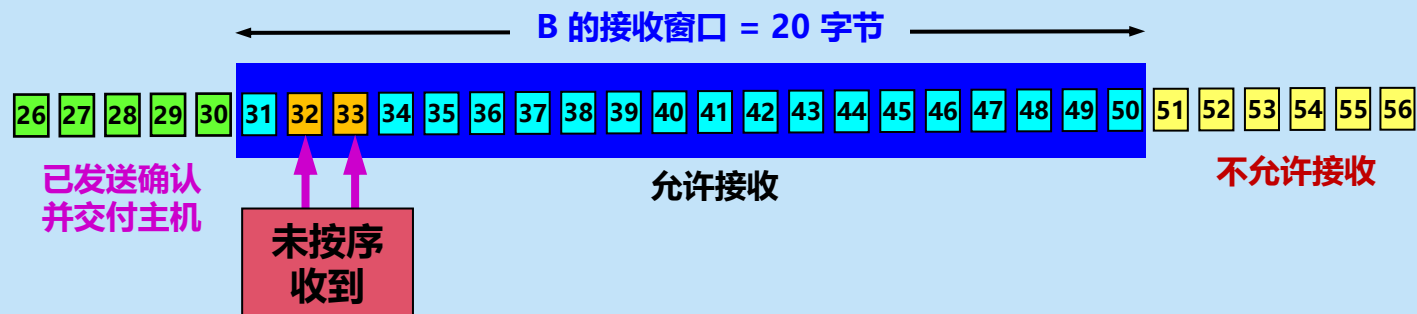
$P_2 - P_1$ = 已发送但尚未收到确认的字节数

$P_3 - P_2$ = 允许发送但尚未发送的字节数 (又称为可用窗口)



TCP可靠传输的实现

接收窗口

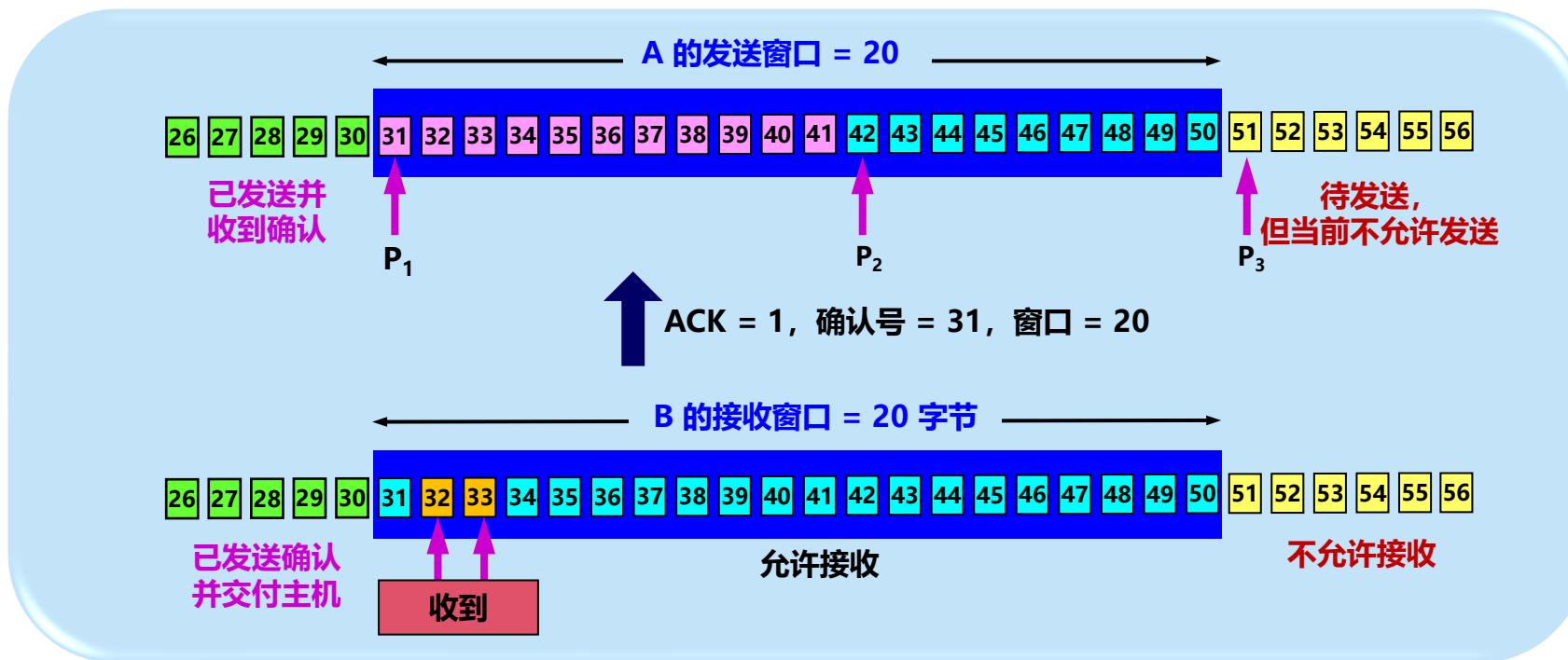


B 收到了序号为 32 和 33 的数据，但未收到序号为 31 的数据。
因此，因此发送的确认报文段中的确认号是 31（即期望收到的序号）。



TCP可靠传输的实现

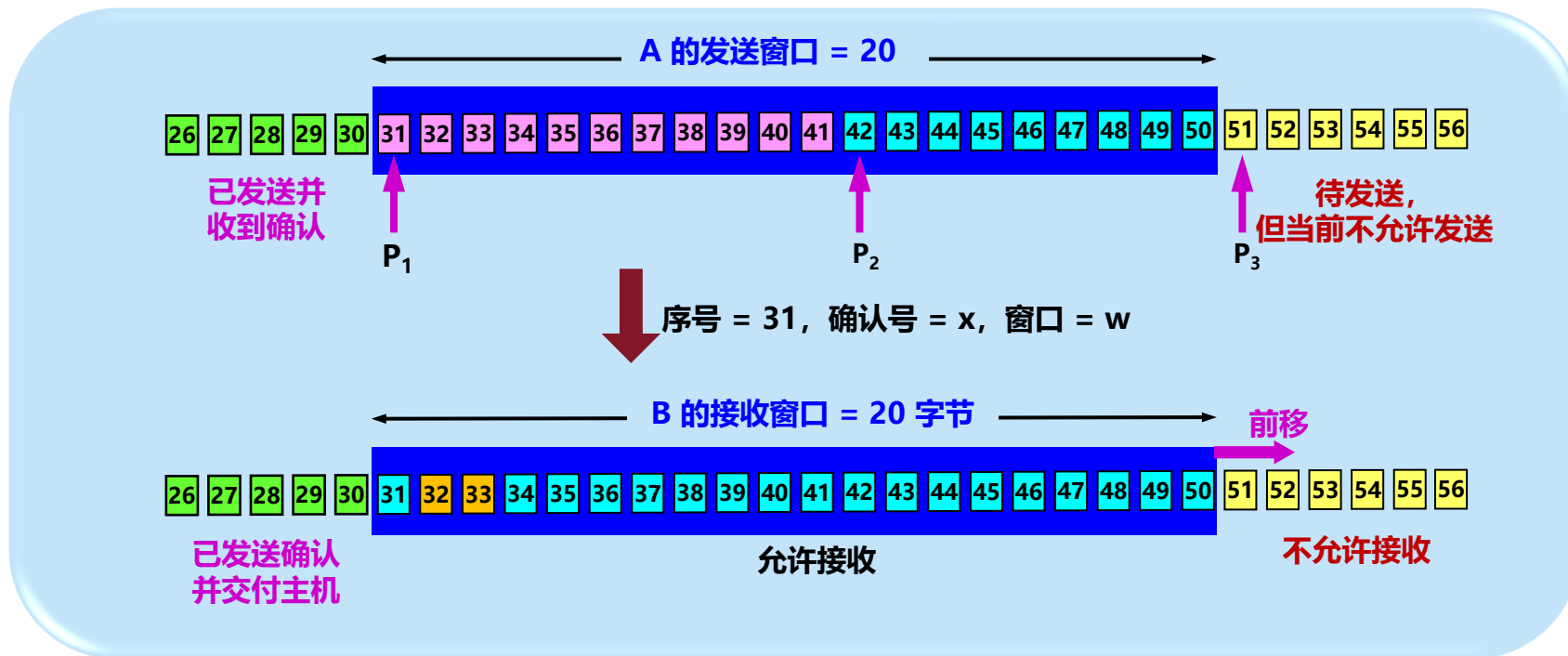
窗口的滑动





TCP可靠传输的实现

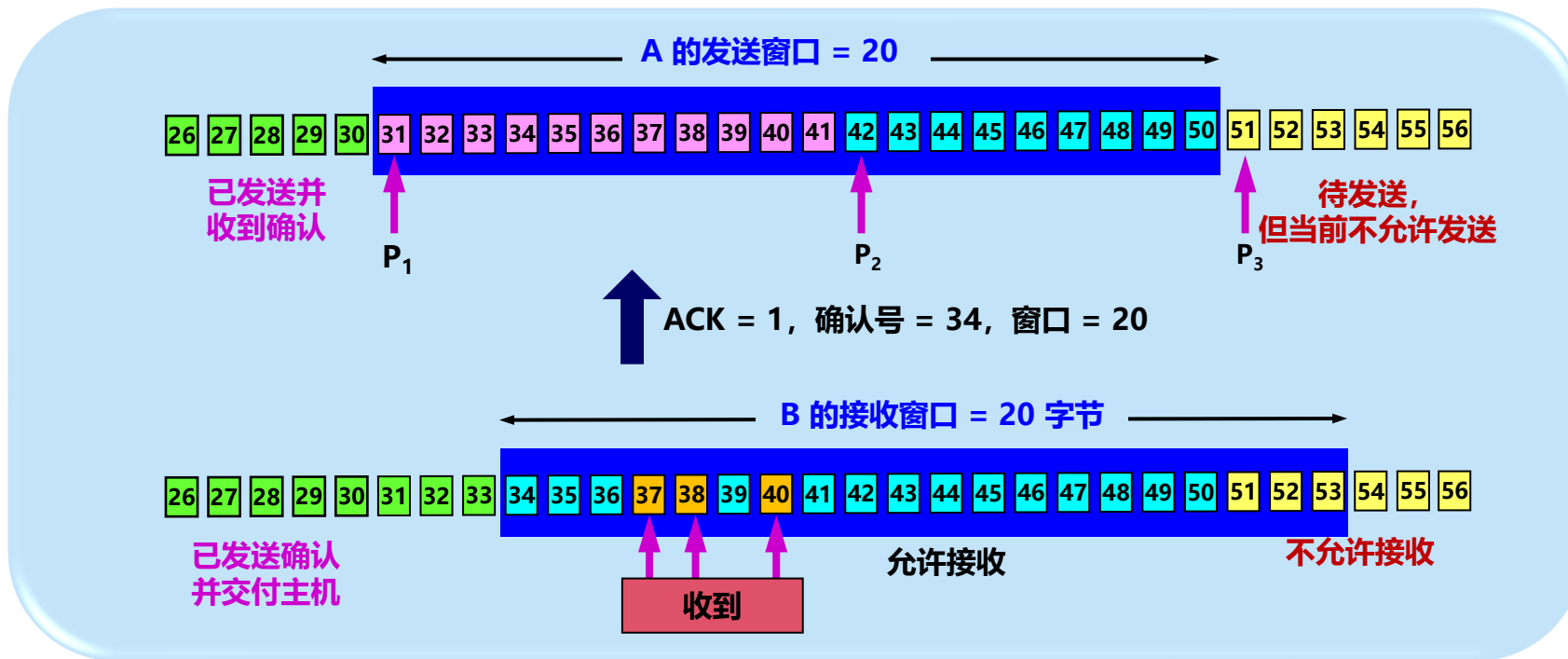
窗口的滑动





TCP可靠传输的实现

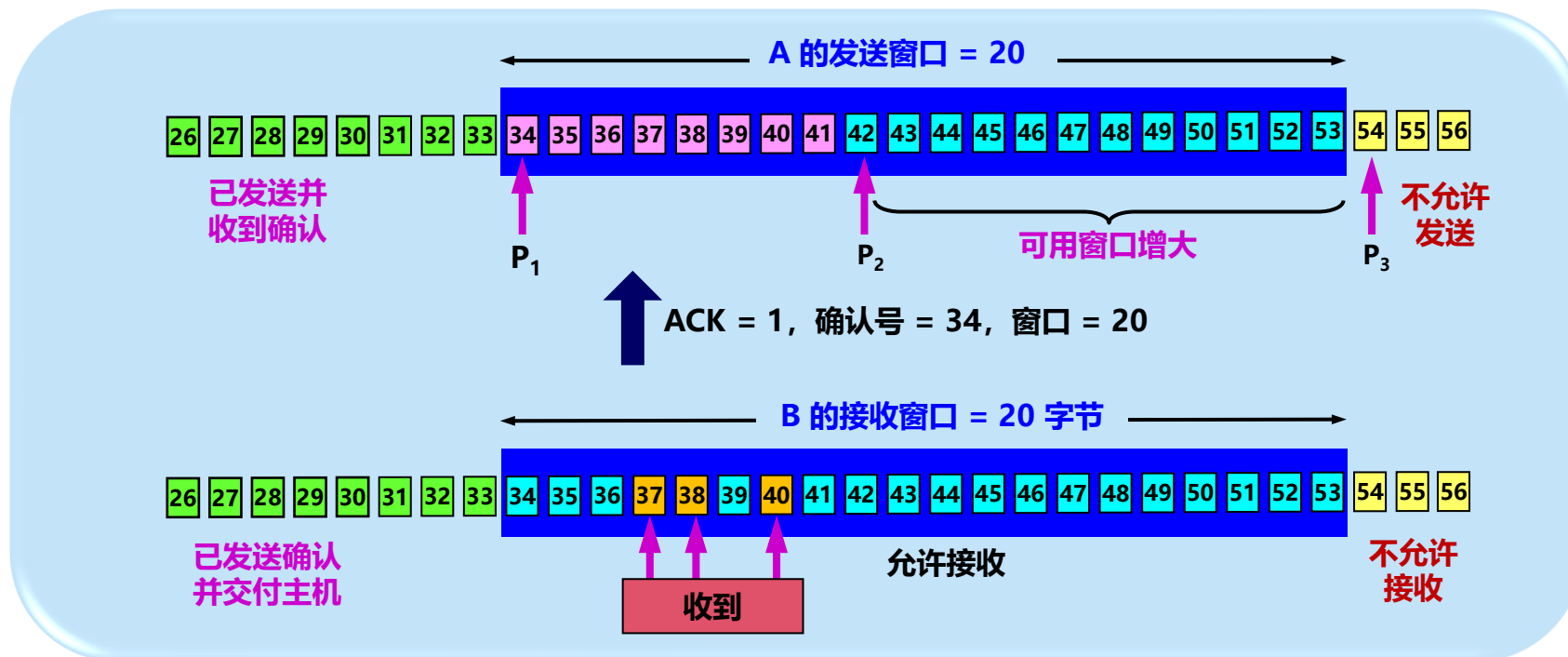
窗口的滑动





TCP可靠传输的实现

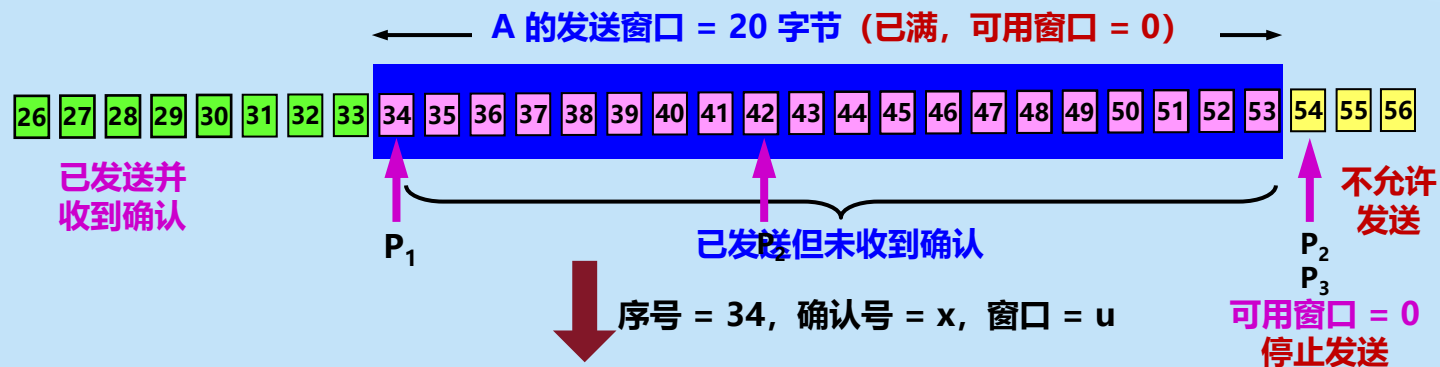
窗口的滑动





TCP可靠传输的实现

窗口的滑动

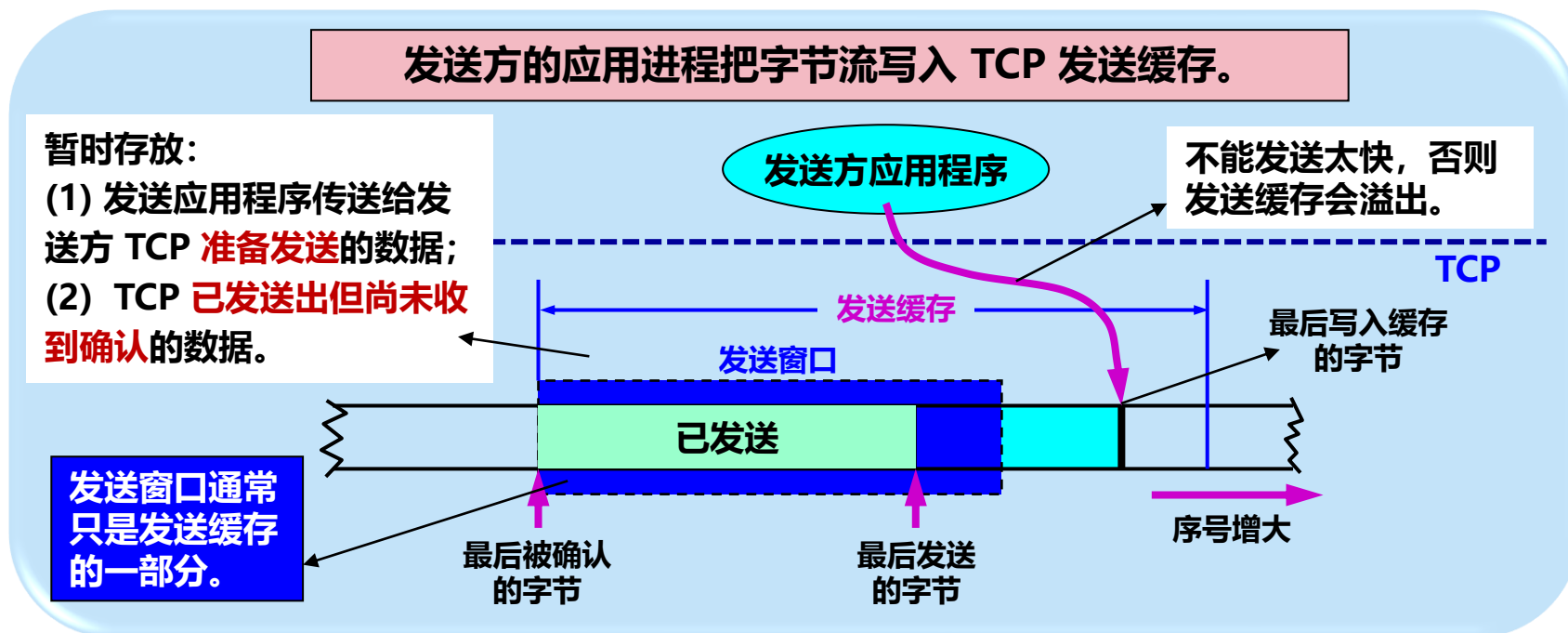


- A 未收到确认的原因有：① B 未发送；② B 已发送，但还未到达 A。
- 为保证可靠传输，A 只能认为 B 还没有收到这些数据。A 经过一段时间后（由超时计时器控制）就重传这部分数据，重新设置超时计时器，直到收到 B 的确认为止。
- 如果 A 按序收到落在发送窗口内的确认号，就使发送窗口向前滑动，并发送新的数据。



TCP可靠传输的实现

发送缓存与发送窗口

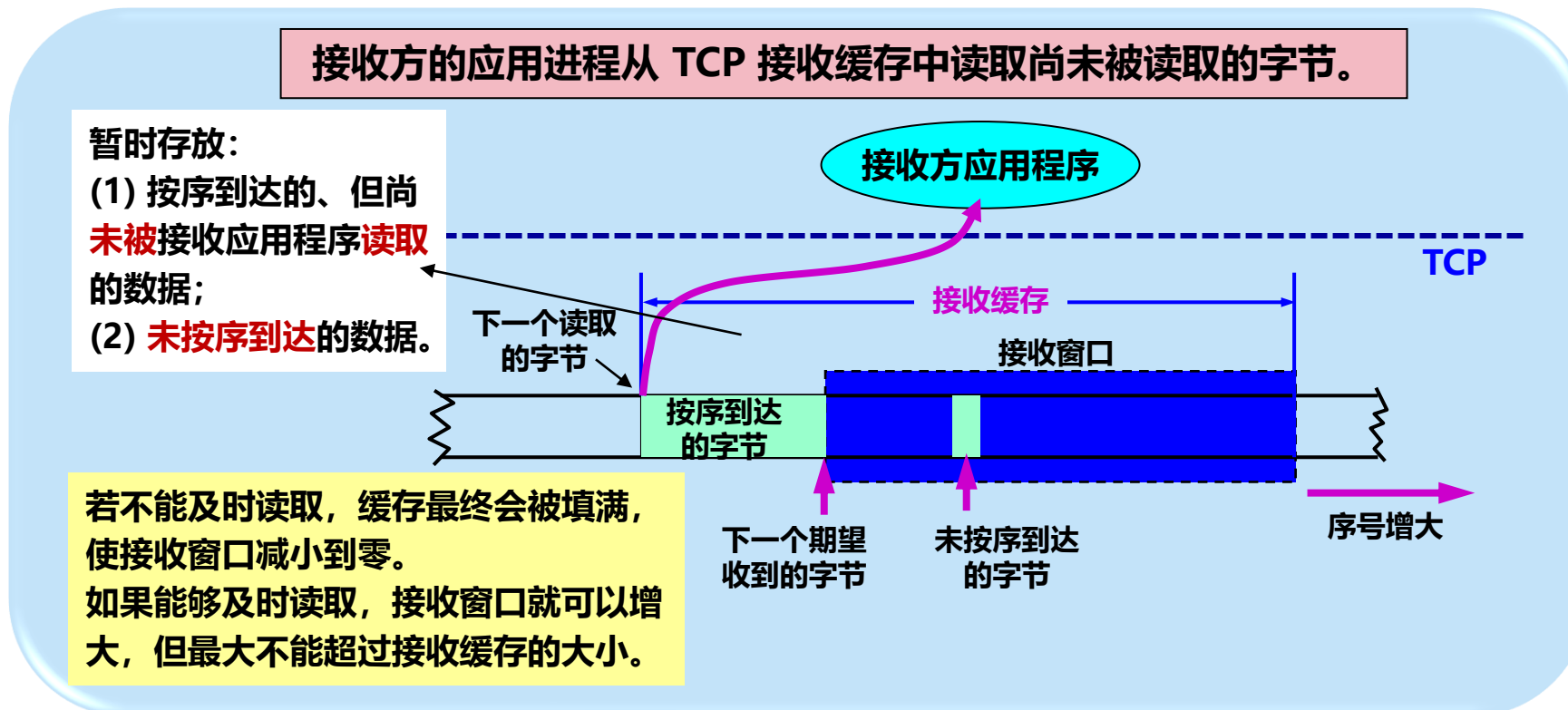


缓存中的字节数 = 发送应用程序最后写入缓存的字节 - 最后被确认的字节



TCP可靠传输的实现

接收缓存与接收窗口





TCP可靠传输的实现

需要强调三点

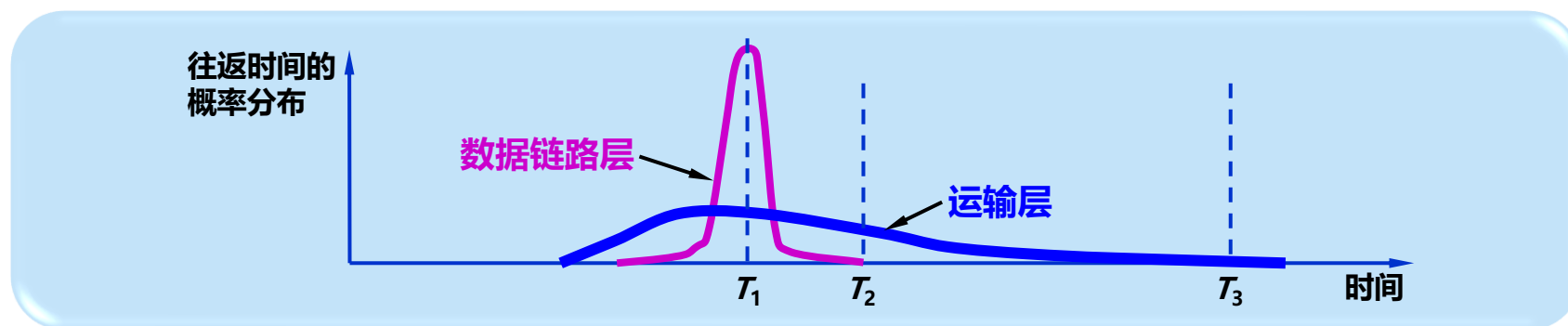
- (1) 发送窗口是根据接收窗口设置的，但在同一时刻，发送窗口并不总是和接收窗口一样大（因为有一定的时间滞后）
- (2) TCP 标准没有规定对不按序到达的数据应如何处理。通常是先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再按序交付上层的应用进程
- (3) TCP 要求接收方必须有累积确认的功能，以减小传输开销。接收方可以在合适的时候发送确认，也可以在自己有数据要发送时把确认信息顺便捎带上。但接收方不应过分推迟发送确认，否则会导致发送方不必要的重传，捎带确认实际上并不经常发生



TCP可靠传输的实现

超时重传时间的选择

- TCP 发送方在规定的时间内没有收到确认就要**重传**已发送的报文段
- 但**重传时间**的选择是 TCP 最复杂的问题之一
- 互联网环境复杂，IP 数据报所选择的路由变化很大，导致运输层的往返时间 (RTT) 的变化也很大





TCP可靠传输的实现

TCP 超时重传时间设置

- **不能太短**，否则会引起很多报文段的不必要的重传，使网络负荷增大
- **不能过长**，会使网络的空闲时间增大，降低了传输效率

TCP 采用了一种**自适应算法**，它记录一个报文段发出的时间，以及收到相应确认的时间。

这两个时间**之差**就是报文段的**往返时间 RTT**。



TCP可靠传输的实现

加权平均往返时间 RTT_S

- 加权平均往返时间 RTT_S 又称为**平滑的往返时间**

$$\text{新的 } RTT_S = (1 - \alpha) \times (\text{旧的 } RTT_S) + \alpha \times (\text{新的 } RTT \text{ 样本}) \quad (5-4)$$

其中, $0 \leq \alpha < 1$ 。

若 $\alpha \rightarrow 0$, 表示 RTT 值更新较慢。

若 $\alpha \rightarrow 1$, 表示 RTT 值更新较快。

RFC 6298 推荐的 α 值为 $1/8$, 即 0.125



TCP可靠传输的实现

超时重传时间RTO

- RTO (Retransmission Time-Out) 应**略大于**加权平均往返时间 RTT_S
- RFC 6298 建议 **RTO**:
 - RTT_D 是 RTT **偏差的加权平均值**

$$RTO = RTT_S + 4 \times RTT_D \quad (5-5)$$

- RFC 6298 建议 **RTT_D** :

$$\text{新的 } RTT_D = (1 - \beta) \times (\text{旧的 } RTT_D) + \beta \times |RTT_S - \text{新的 RTT 样本}| \quad (5-6)$$

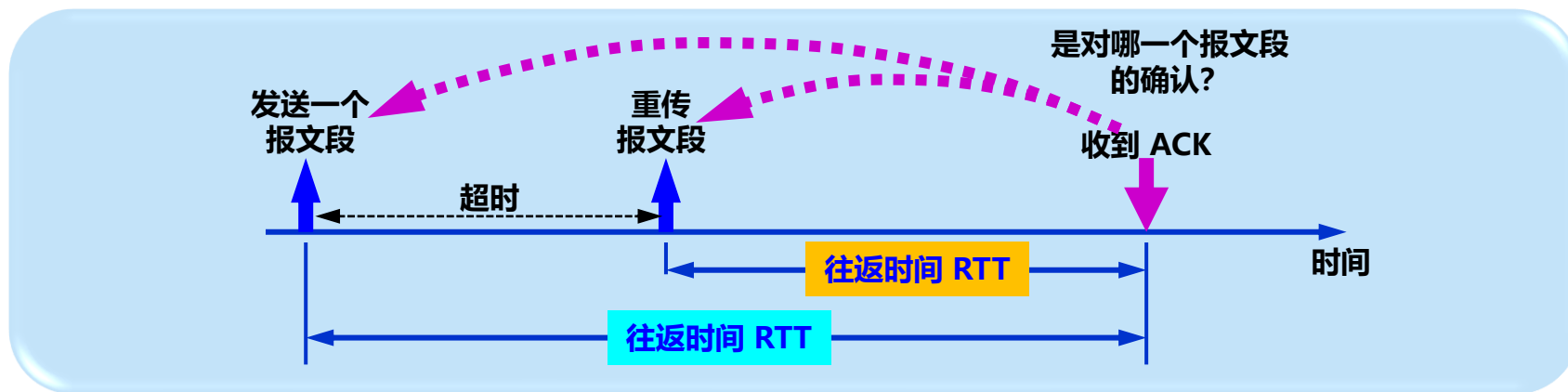
- β 是个小于 1 的系数, 其推荐值是 **1/4**, 即 0.25
- 第一次测量时, **RTT_D** 取测量到的RTT样本值的一半



TCP可靠传输的实现

往返时间RTT的测量相当复杂

- 超时重传报文段后，如何判定此确认报文段是对原来的报文段的确认，还是对重传报文段的确认





TCP可靠传输的实现

Karn 算法

- 在计算平均往返时间 RTT 时，只要报文段**重传**了，就**不采用**其往返时间样本
- **新问题：**当报文段的时延突然**增大**很多时，在原来得出的重传时间内，不会收到确认报文段，于是就重传报文段。但根据 Karn 算法，不考虑重传的报文段的往返时间样本。这样，**超时重传时间就无法更新，造成很多不必要的重传**



TCP可靠传输的实现

修正的Karn 算法

- 报文段每重传一次，就把 RTO 增大一些

$$\text{新的 RTO} = \gamma \times (\text{旧的 RTO})$$

- 系数 γ 的典型值 = 2

- 当**不再发生**报文段的重传时，才根据报文段的往返时延**更新**平均往返时延 RTT 和超时重传时间 RTO 的数值



TCP可靠传输的实现

选择确认SACK

- **问题：**若收到的报文段无差错，只是未按序号，中间还缺少一些序号的数据，那么能否设法**只传送缺少的数据**而不重传已经正确到达接收方的数据
- **解决：****选择确认SACK** (Selective ACK)

假设最大报文段长度 MSS = 5000 字节，起始序号 = 1。

接收方收到了和前面的字节流不连续的两个字节块，缺少序号 1001 ~ 1500、3001 ~ 3500、4501~5000 的字节。



确认报文: ACK = 1, 确认号 = 1001, 窗口 = 6000



TCP可靠传输的实现

RFC 2018 对 SACK (4) 的规定

- 如果要使用**选择确认**，在建立 TCP 连接时，要在 TCP 首部的选项中加上**允许 SACK** 选项，且双方必须事先商定好
- 如果使用选择确认，原来首部中的**确认号**的用法仍然不变（累积确认）。只是在 TCP 首部中都增加了 **SACK 选项**，以便报告收到的**不连续的**字节块的**边界**

类别 = 4	长度 = 2
--------	--------

允许 SACK 选项

类别 = 5	长度 = ?
--------	--------

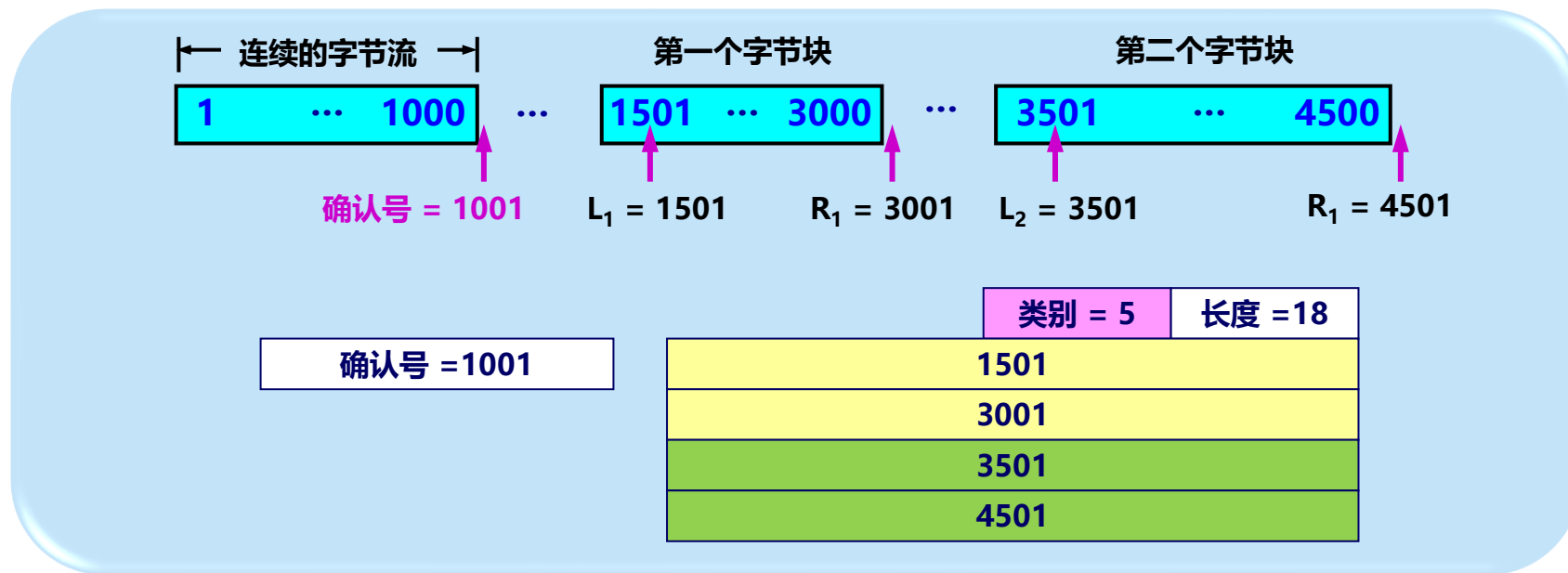
第 1 个字节块的左边界	最多 4 个 字节块
第 1 个字节块的右边界	
第 2 个字节块的左边界	
第 2 个字节块的右边界	
第 3 个字节块的左边界	
第 3 个字节块的右边界	
第 4 个字节块的左边界	
第 4 个字节块的右边界	

SACK 选项



TCP可靠传输的实现

SACK



左边界 = 第一个字节的序号, 右边界 = 最后一个字节序号 + 1。



TCP可靠传输的实现

SACK





TCP可靠传输的实现

练习

- 主机A向主机 B 连续发送了两个TCP 报文段，其序号分别是 70 和 100。试问
 - (1) 第一个报文段携带了多少字节的数据？
 - (2) 主机 B 收到第一个报文段后，发回的确认中的确认号应当是多少？
 - (3) 如果 B 收到第二个报文段后，发回的确认中的确认号是 180，试问A 发送的第二个报文段中的数据有多少字节？
 - (4) 如果 A 发送的第一个报文段丢失了，但第二个报文段到达了 B。B 在第二个报文段到达后向 A 发送确认。试问这个确认号应为多少？
- 答: (1) 30字节 (2) 100 (3) 80 (4) 70



章节大纲

1	运输层协议概述
2	用户数据报协议UDP
3	传输控制协议TCP概述
4	可靠传输的工作原理
5	TCP报文段的首部格式
6	TCP可靠传输的实现
7	TCP 的流量控制
8	TCP的拥塞控制
9	TCP的运输连接管理



TCP的流量控制

思考

- 为什么要进行流量控制?
- TCP的流量控制如何实现?



TCP的流量控制

利用滑动窗口实现流量控制

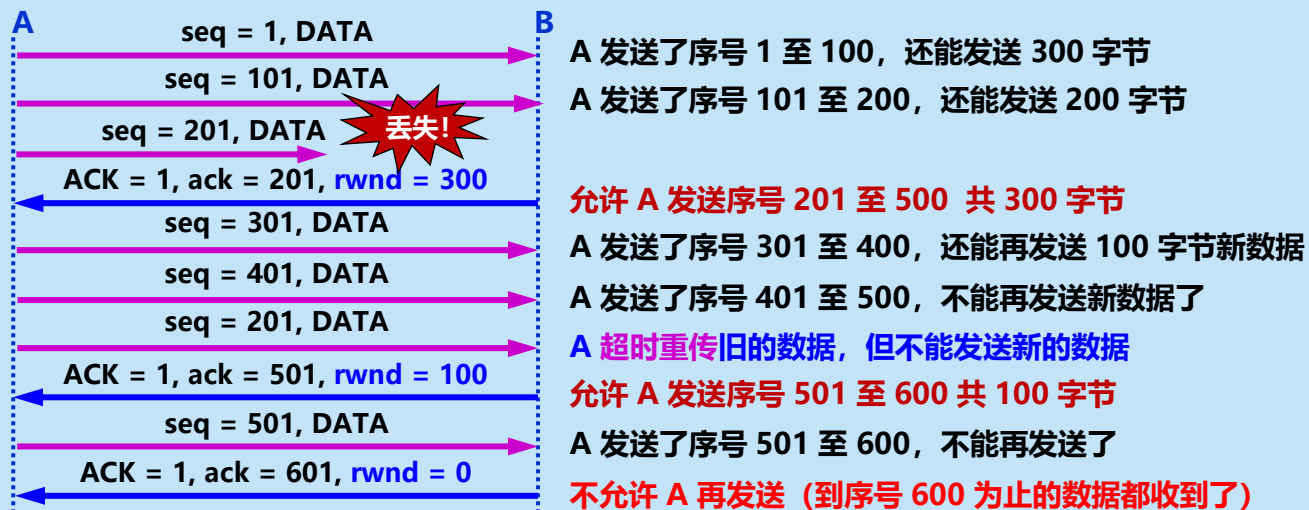
- **流量控制 (flow control)**：让发送方的发送速率不要太快，使接收方来得及接收
- TCP利用**滑动窗口机制**，可以在 TCP 连接上实现对**发送方**的流量控制



TCP的流量控制

利用可变窗口进行流量控制举例

A 向 B 发送数据，MSS = 100 字节。
在连接建立时，B 告诉 A：“我的接收窗口 $rwnd = 400$ (字节)”。

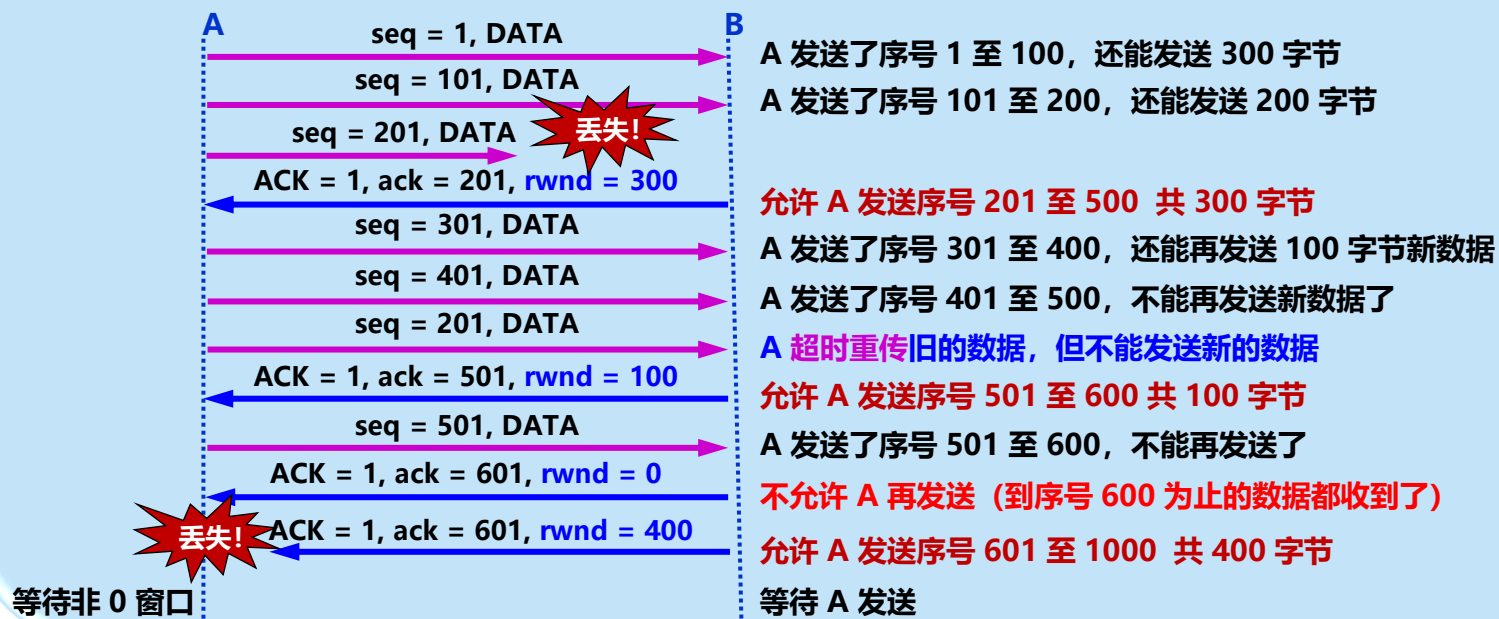




TCP的流量控制

可能发死锁

A 向 B 发送数据, MSS = 100 字节。
在连接建立时, B 告诉 A: “我的接收窗口 $rwnd = 400$ (字节)”。





TCP的流量控制

持续计时器

- **持续计时器** (persistence timer): 只要 TCP 连接的一方收到对方的**零窗口**通知, 就启动该持续计时器
 - 若持续计时器设置的时间到期, 就发送一个**零窗口探测报文段** (仅携带 1 字节的数据), 对方在确认这个探测报文段时给出当前窗口值
 - 若窗口仍然是零, 收到这个报文段的一方就重新设置持续计时器
 - 若窗口不是零, 则死锁的僵局就可以打破了



TCP的流量控制

TCP的传输效率

- 控制TCP发送报文段的时机：**三种机制**

- TCP 维持一个变量，它等于最大报文段长度 MSS。只要缓存中存放的数据达到 MSS 字节时，就组装成一个 TCP 报文段发送出去
- 由发送方的应用进程指明要求发送报文段，即 TCP 支持的**推送** (push) 操作
- 发送方的一个计时器期限到了，这时就把当前已有的缓存数据装入报文段（但长度不能超过 MSS）发送出去

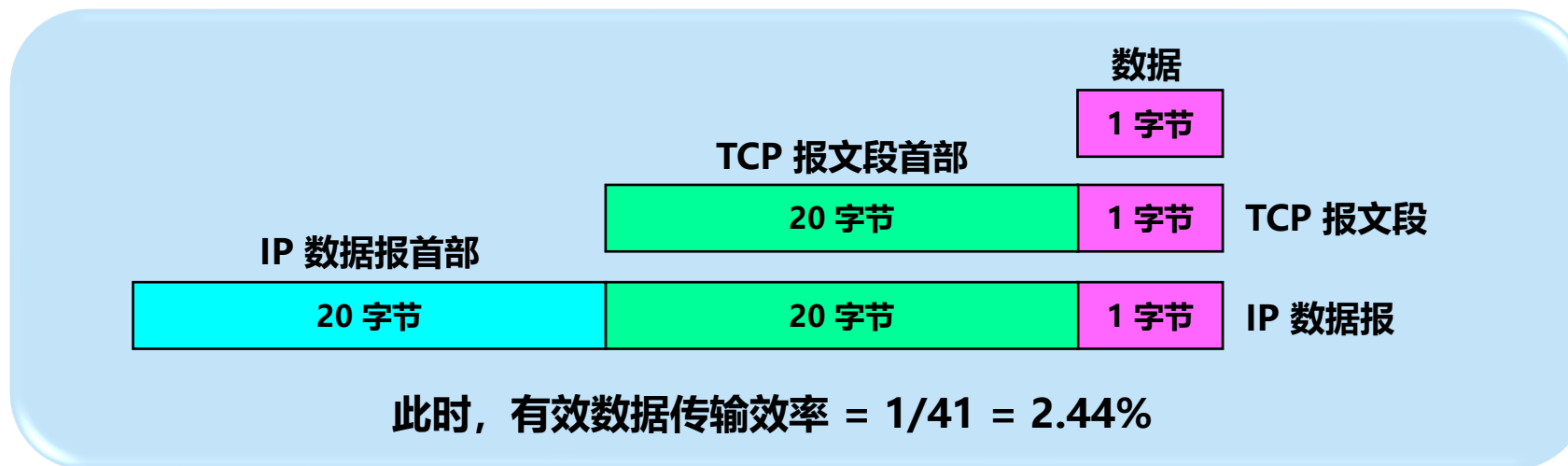
如何控制 TCP 发送报文段的时机仍然是一个较为复杂的问题。



TCP的流量控制

糊涂窗口综合症

- **糊涂窗口综合症：** 每次仅发送一个字节或很少几个字节的数据时，有效数据传输效率变得很低的现象

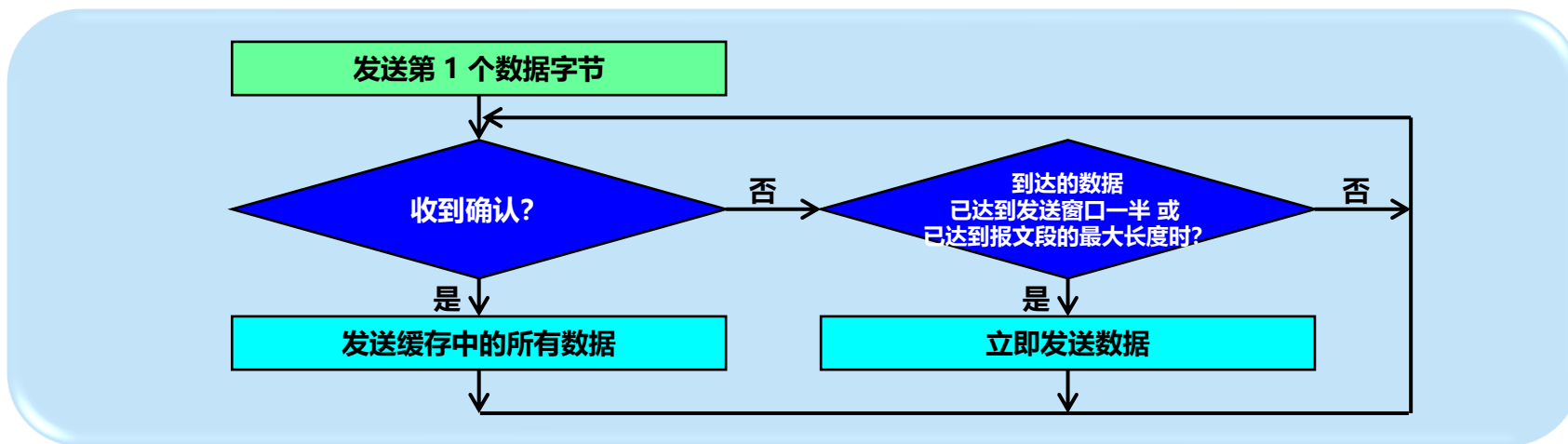




TCP的流量控制

发送方糊涂窗口综合症

- 发送方 TCP **每次接收到一字节**的数据后就发送
- 发送一个字节需要形成 41 字节长的 IP 数据报。效率很低
- **解决方法**: 使用 Nagle 算法

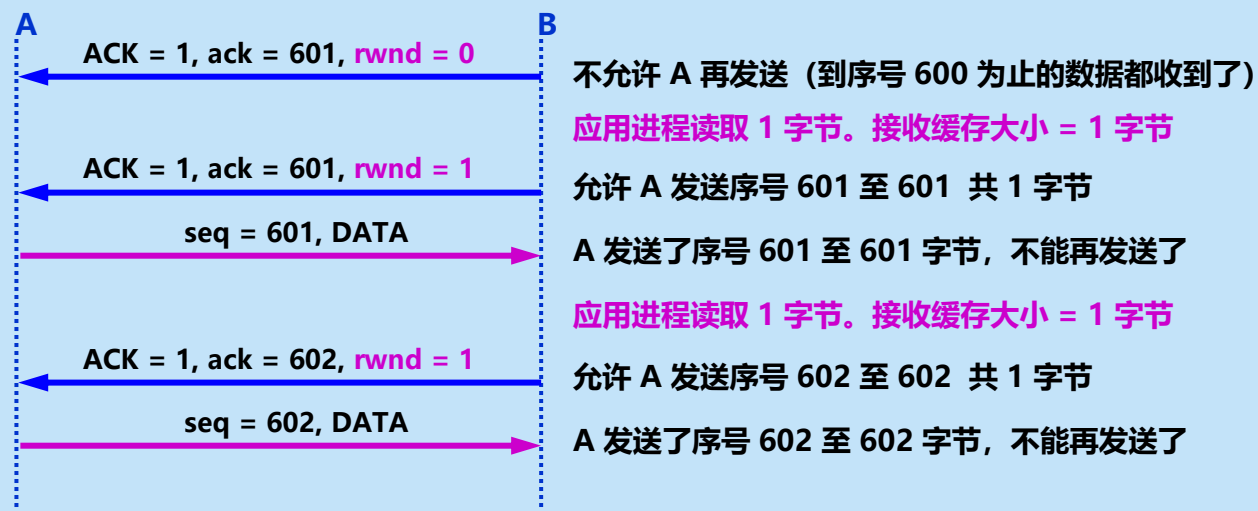




TCP的流量控制

接收方糊涂窗口综合症

原因：接收方应用进程消耗数据太慢，例如：每次只读取一个字节。





TCP的流量控制

接收方糊涂窗口综合症

- **解决方法**：让接收方等待一段时间，使得或者接收缓存**已有足够空间**容纳一个最长的报文段，或者等到接收缓存**已有一半空闲的空间**。只要出现这两种情况之一，接收方就发出确认报文，并向发送方通知当前的窗口大小

上述两种方法可配合使用，使得在发送方不发送很小的报文段的同时，接收方也不要再在缓存刚刚有了一点小的空间就急忙把这个很小的窗口大小信息通知给发送方。



章节大纲

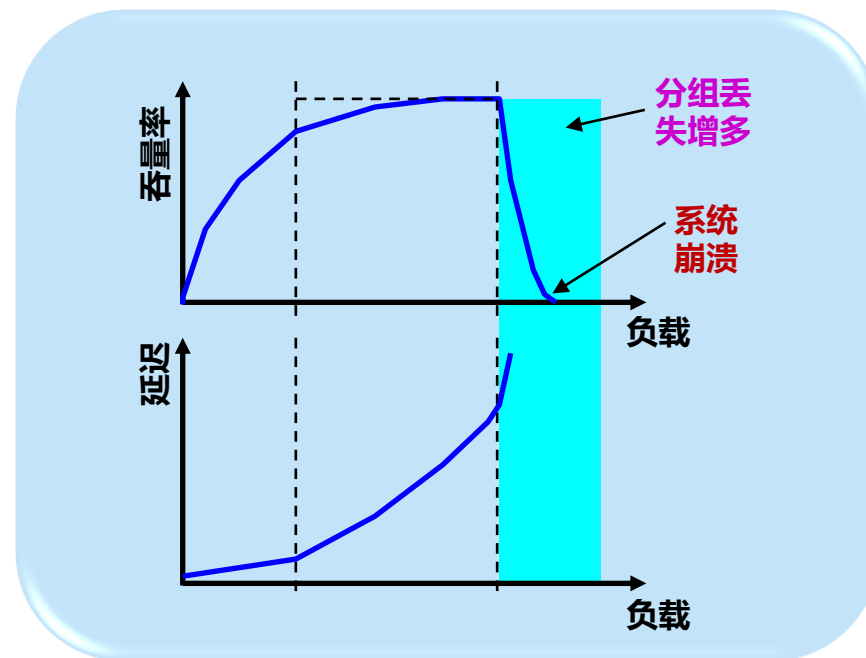
1	运输层协议概述
2	用户数据报协议UDP
3	传输控制协议TCP概述
4	可靠传输的工作原理
5	TCP报文段的首部格式
6	TCP可靠传输的实现
7	TCP 的流量控制
8	TCP的拥塞控制
9	TCP的运输连接管理



TCP的拥塞控制

拥塞控制的一般原理

- 在某段时间，若对网络中某**资源的需求超过了该资源所能提供的可用部分**，网络的性能就要明显变坏，整个网络的吞吐量将随输入负荷的增大而下降。这种现象称为**拥塞** (congestion)
- 最坏结果：**系统崩溃**





TCP的拥塞控制

拥塞产生的原因

- 由许多因素引起。例如
 - 节点缓存容量太小，丢弃达到的分组
 - 链路容量不足，排队等待时间增加
 - 处理机处理速率太慢，排队等待时间增加
 - 拥塞本身会进一步加剧拥塞，拥塞趋于恶化
- 出现网络拥塞的条件

$$\sum \text{对资源需求} > \text{可用资源} \quad (5-7)$$



TCP的拥塞控制

增加资源能解决拥塞吗？

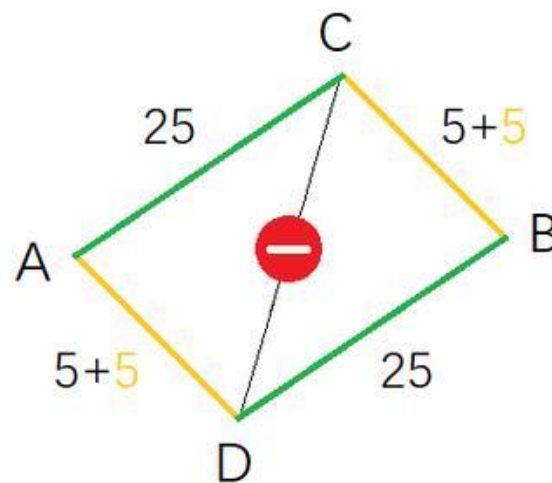
- 不能，而且还可能使网络的性能更坏
- 例如：
 - 增大缓存，但未提高输出链路的容量和处理机的速度？
 - 带来的问题：排队等待时间将会大大增加，引起大量超时重传，解决不了网络拥塞
 - 提高处理机处理的速率？
 - 带来的问题：会将瓶颈转移到其他地方
 - 拥塞引起的**重传**并不会缓解网络的拥塞，反而会加剧网络的拥塞



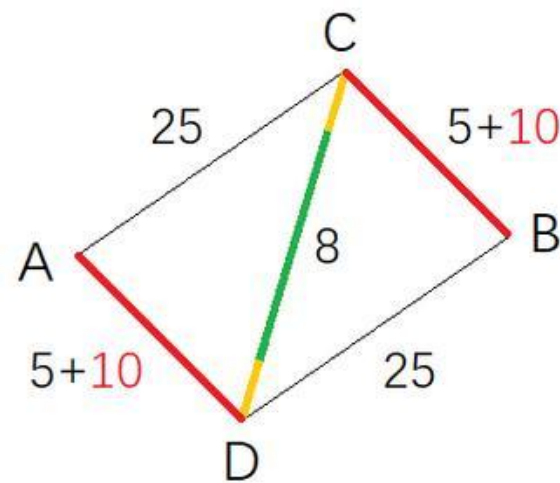
TCP的拥塞控制

布雷斯悖论 (Braess悖论)

- 在由自私个体组成的交通网络中，增加新的道路或通行能力，反而可能导致所有出行者的旅行时间增加，从而降低整个网络的效率
 - 1969年，德国斯图加特市为缓解市中心拥堵而新建一条街道，结果交通状况反而恶化，在拆除该道路后交通得到改善
 - 1990年，美国纽约市在地球日临时关闭繁忙的第42街，意外地改善了周边交通流
 - 2003年至2005年，韩国首尔实施清溪川修复项目，拆除一条六车道高速公路后，不仅环境得到改善，城市交通速度也显著提升



A 到 B 用时 35 分钟
(CD 禁止通行)



A 到 B 用时 38 分钟
(CD 通车)



TCP的拥塞控制

拥塞控制与流量控制的区别

拥塞控制

- 防止过多的数据注入到网络中，避免网络中的路由器或链路过载。
- 是一个全局性的过程，涉及到所有的主机、路由器，以及与降低网络传输性能有关的所有因素。

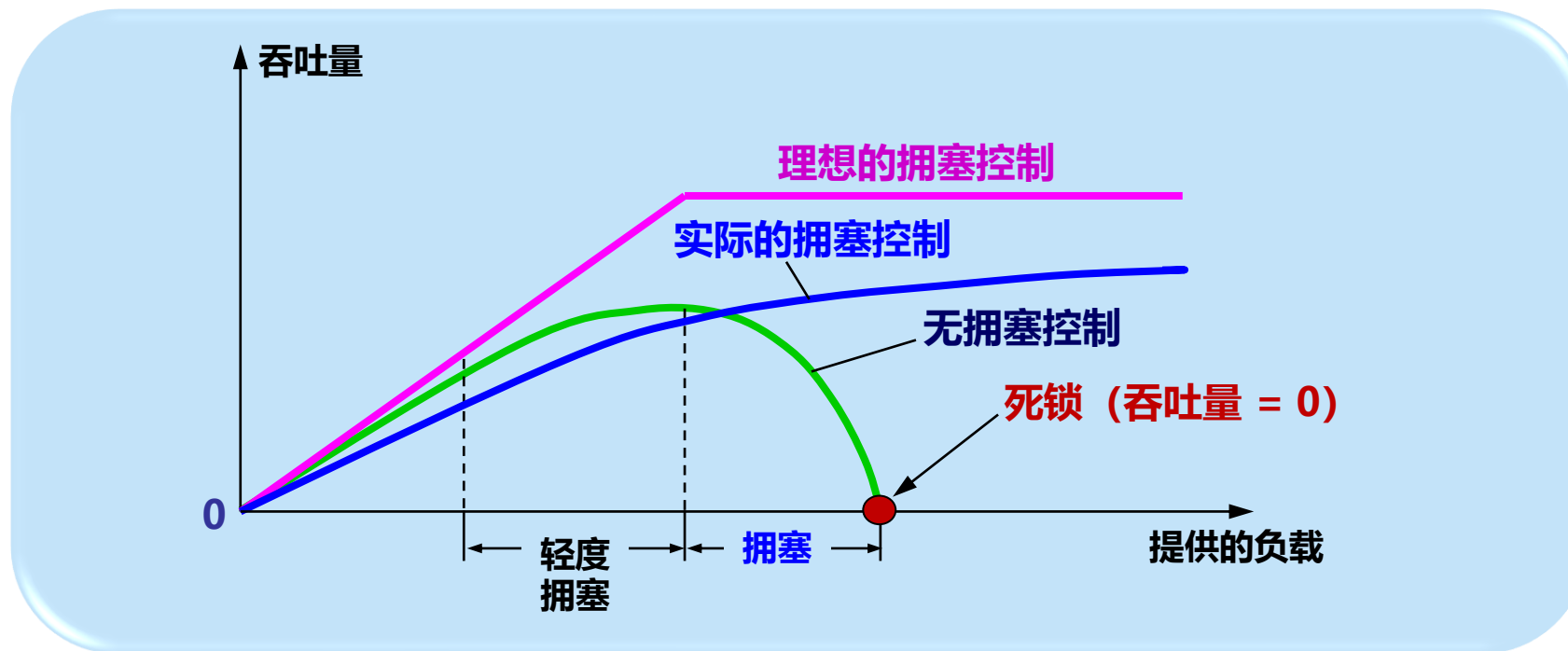
流量控制

- 抑制发送端发送数据的速率，以使接收端来得及接收。
- 点对点通信量的控制，是个端到端的问题。



TCP的拥塞控制

拥塞控制所起的作用





TCP的拥塞控制

拥塞控制的一般原理

- 拥塞控制的前提：网络能够**承受**现有的网络负荷
- 实践证明，拥塞控制是很难设计的，因为它是一个**动态问题**
- 分组的丢失是网络发生拥塞的**征兆**，而不是原因
- 在许多情况下，甚至正是**拥塞控制本身**成为引起网络性能恶化、甚至发生死锁的原因



TCP的拥塞控制

开环控制和闭环控制

开环控制

- 在设计网络时，事先**考虑周全**，力求工作时不发生拥塞。
- **思路**：力争**避免**发生拥塞。
- 但一旦整个系统运行起来，就不再中途进行改正了。

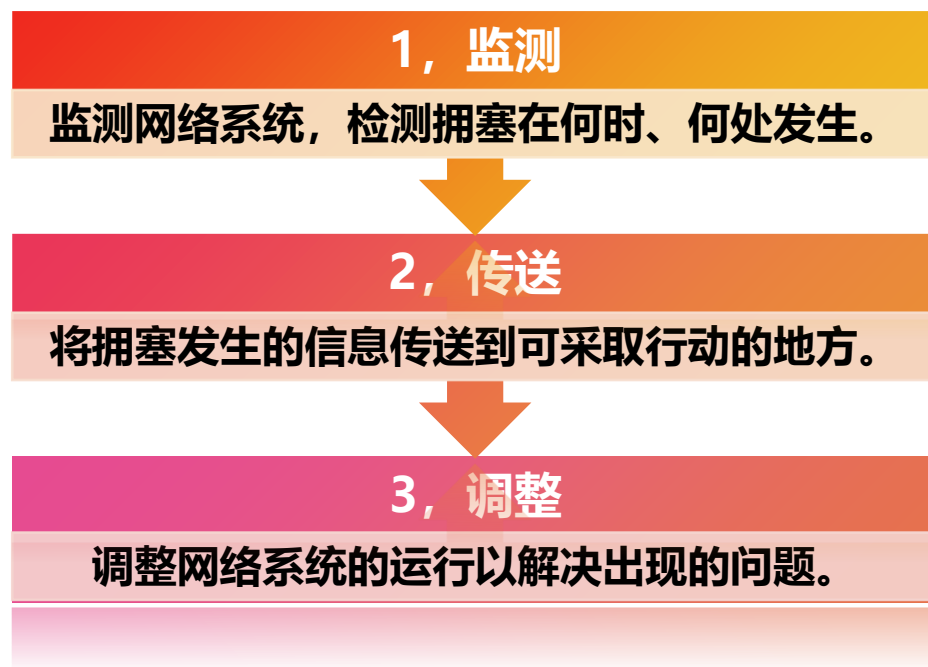
闭环控制

- 基于反馈环路的概念。
- 根据网络**当前运行状态**采取相应控制措施。
- **思路**：在发生拥塞后，采取措施进行控制，**消除**拥塞。



TCP的拥塞控制

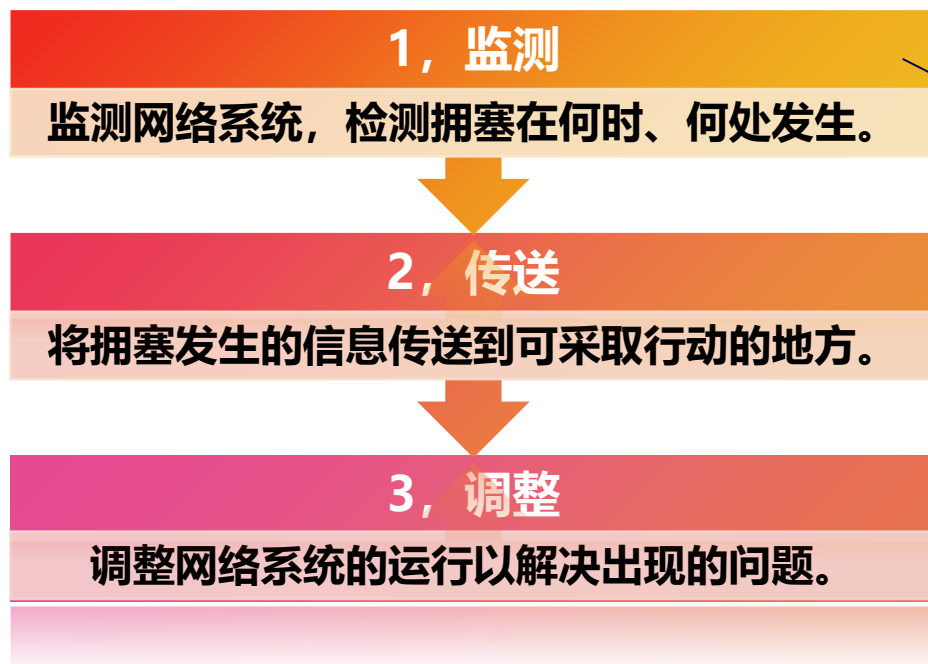
闭环控制措施





TCP的拥塞控制

闭环控制措施



- 主要指标有：

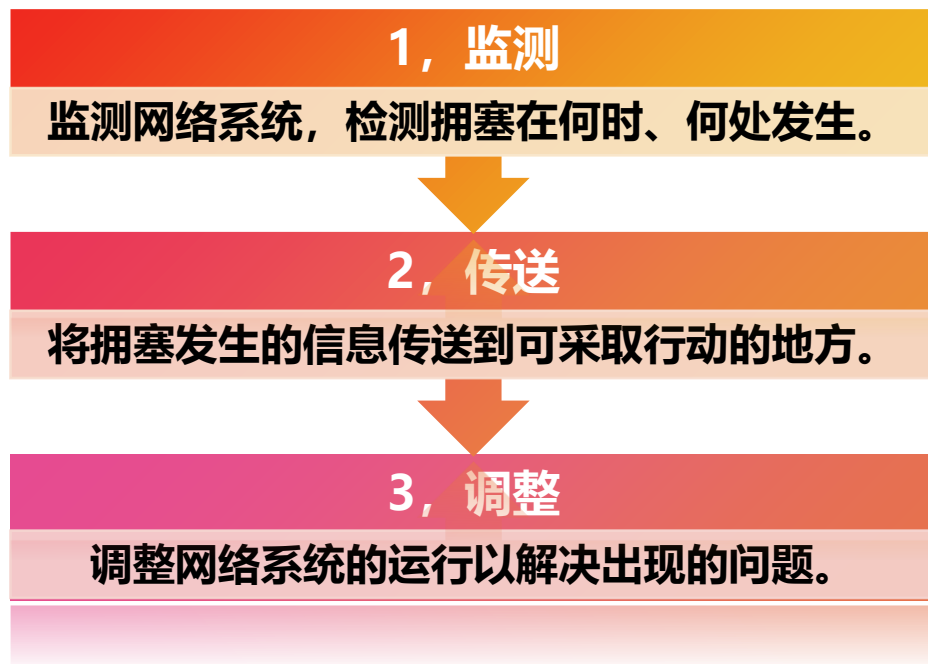
1. 由于缺少缓存空间而被丢弃的分组的百分数；
2. 平均队列长度；
3. 超时重传的分组数；
4. 平均分组时延；
5. 分组时延的标准差，等等。

- 这些指标的上升都标志着拥塞的增长。



TCP的拥塞控制

闭环控制措施

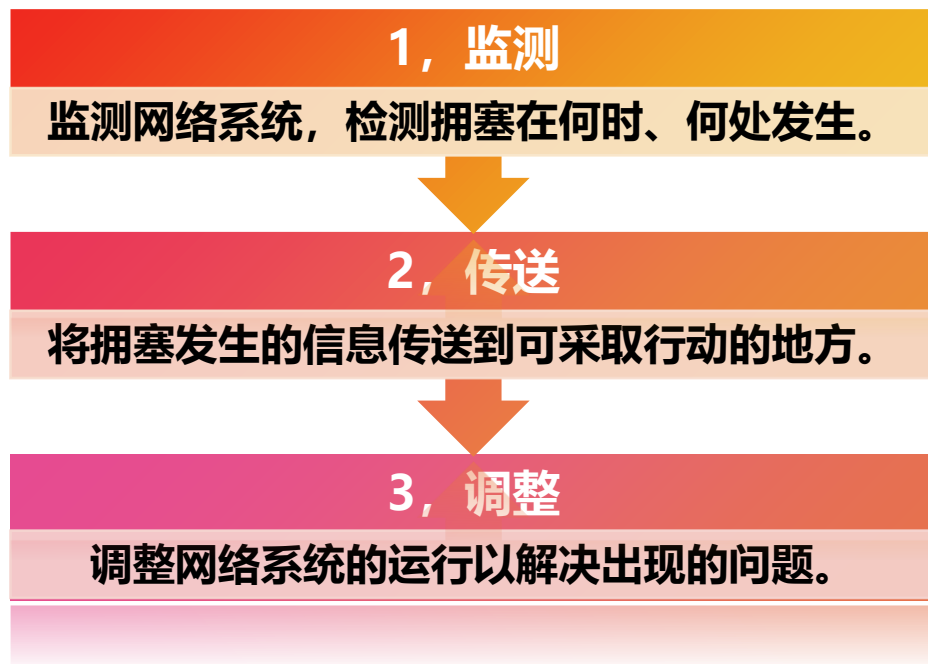


1. 将拥塞发生的信息传送到产生分组的源站。
2. 在路由器转发的分组中保留一个比特或字段，用该比特或字段的值表示网络没有拥塞或产生了拥塞。
3. 主机或路由器周期性地发出探测分组询问拥塞是否发生等。



TCP的拥塞控制

闭环控制措施



- 过于频繁，会使系统产生不稳定的振荡。
- 过于迟缓，不具有任何实用价值。
- 选择正确的时间常数是相当困难的。



TCP的拥塞控制

TCP的拥塞控制方法

- TCP 采用基于**滑动窗口的方法**进行拥塞控制，属于闭环控制方法
- TCP 发送方维持一个**拥塞窗口 cwnd** (Congestion Window)
- **拥塞窗口**的大小取决于网络的拥塞程度，并且是**动态变化**的
- 发送端利用**拥塞窗口**根据网络的拥塞情况调整发送的数据量
- 综合考虑流量控制和拥塞控制时，发送窗口大小不仅取决于接收方窗口，还取决于网络的拥塞状况
- **真正的发送窗口值：**

真正的发送窗口值 = Min (接收方通知的窗口值, 拥塞窗口值)



TCP的拥塞控制

控制拥塞窗口变化的原则

- 只要网络没有出现拥塞，拥塞窗口就可以再增大一些，以便把更多的分组发送出去，提高网络的利用率
- 但只要网络出现拥塞或有可能出现拥塞，就必须把拥塞窗口减小一些，以减少注入到网络中的分组数，缓解网络出现的拥塞



TCP的拥塞控制

发送方判断拥塞的方法：隐式反馈

- 因传输出差错而丢弃分组的**概率很小**（远小于1 %）
- 因此，发送方在超时重传计时器启动时，**就判断网络出现了拥塞**

超时重传计时器超时

• 网络已经出现了拥塞。

收到 3 个重复的确认

• 预示网络可能会出现拥塞。



TCP的拥塞控制

TCP拥塞控制算法

- 四种拥塞控制算法（RFC 5681）：
 - **慢开始** (slow-start)
 - **拥塞避免** (congestion avoidance)
 - **快重传** (fast retransmit)
 - **快恢复** (fast recovery)



TCP的拥塞控制

慢开始 (slow start)

- **目的：** 探测网络的负载能力或拥塞程度
- **算法：** 由小到大逐渐增大注入到网络中的数据字节，即：由小到大逐渐**增大拥塞窗口**数值
- **2 个控制变量**
 - **拥塞窗口cwnd** (congestion window)
 - 初始值两种设置方法：1) 1 至 2 个最大报文段 MSS (旧标准)
2) 2 至 4 个最大报文段 MSS (RFC 5681)
 - **慢开始门限** (ssthresh)
 - 防止拥塞窗口增长过大引起网络拥塞



TCP的拥塞控制

慢开始 (slow start)

- **拥塞控制窗口增大**：在每收到一个**对新的报文段的确认**，就把拥塞窗口增加最多一个**发送方的最大报文段 SMSS** (Sender Maximum Segment Size) 的数值

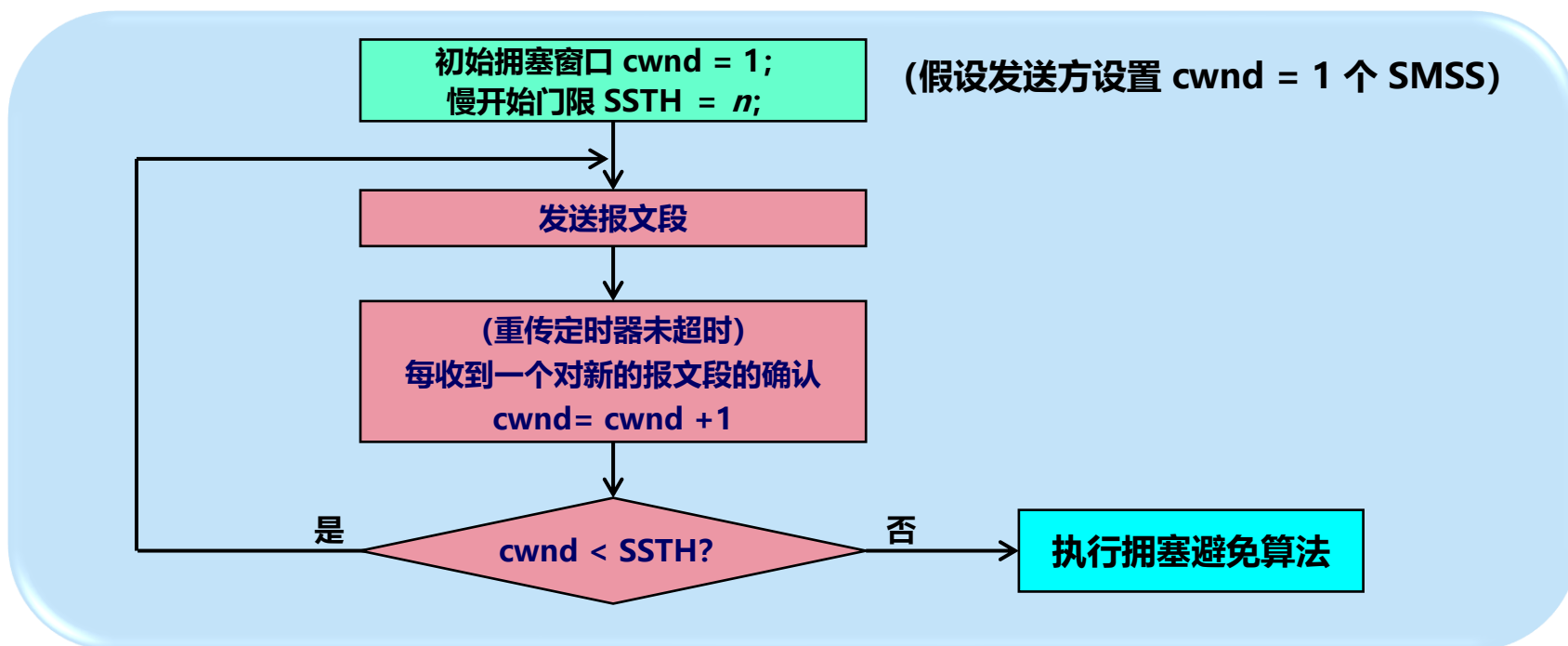
$$\text{拥塞窗口 cwnd 每次的增加量} = \min(N, \text{SMSS}) \quad (5-8)$$

其中 N 是原先未被确认的、但现在被刚收到的确认报文段所确认的字节数



TCP的拥塞控制

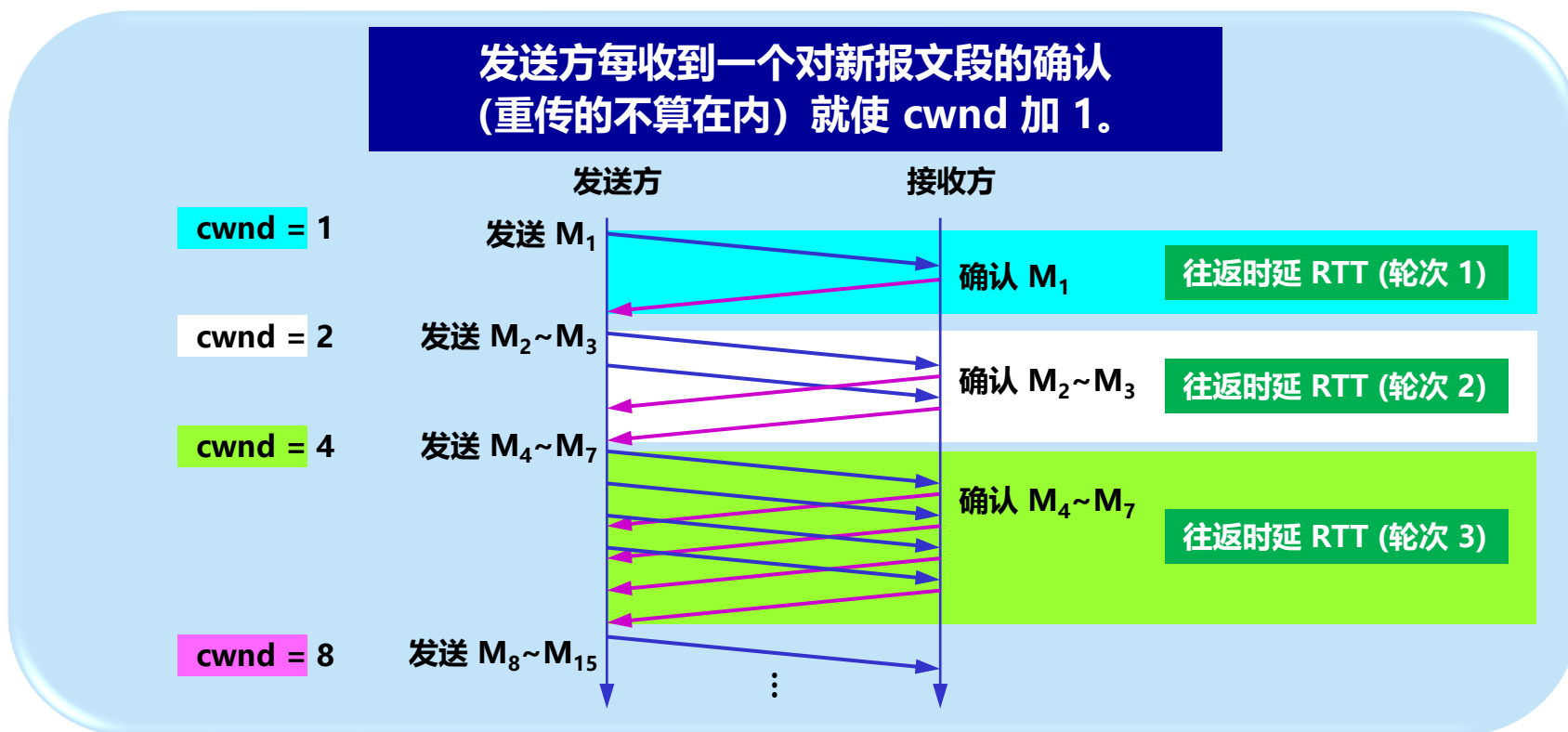
慢开始 (slow start)





TCP的拥塞控制

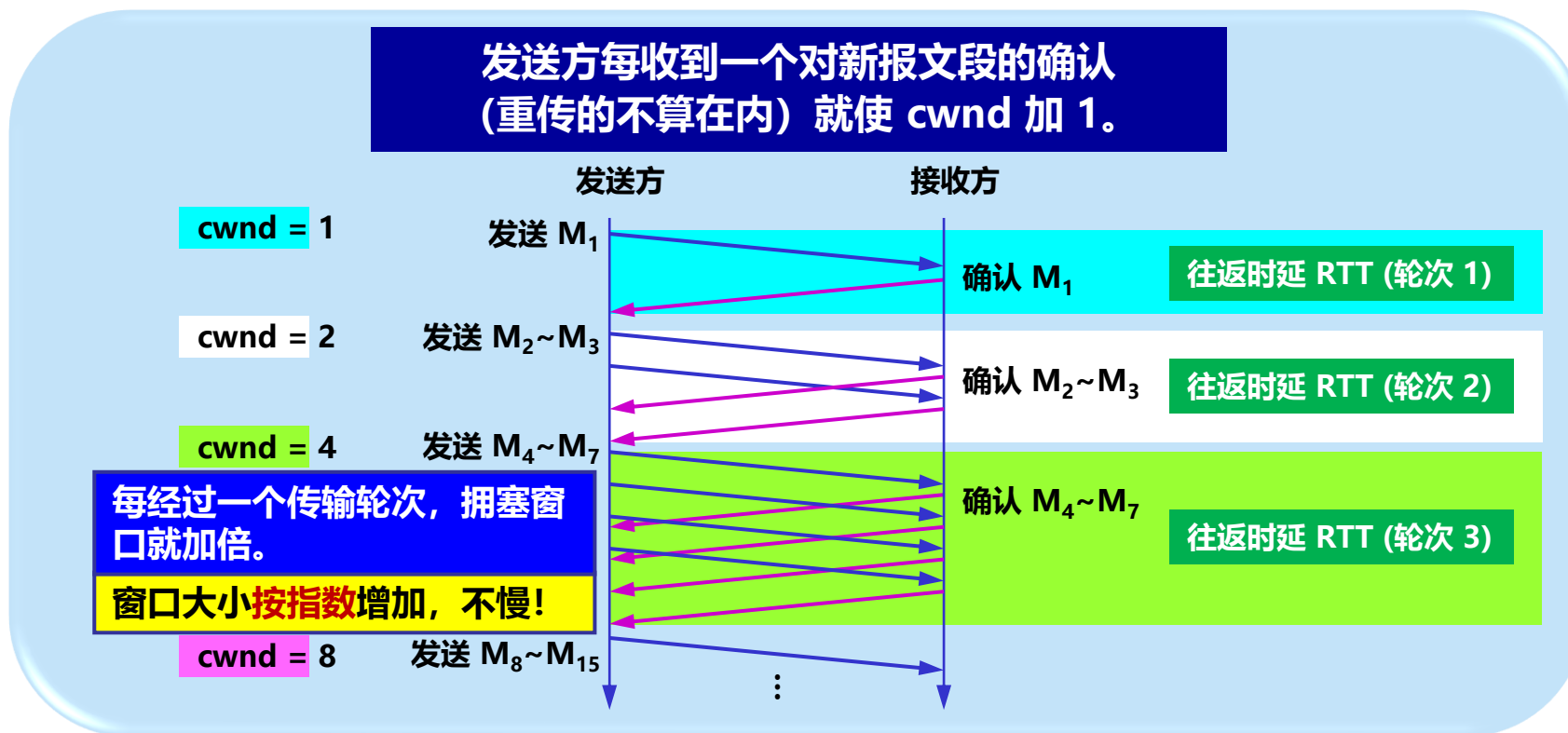
慢开始 (slow start)





TCP的拥塞控制

慢开始 (slow start)





TCP的拥塞控制

传输轮次

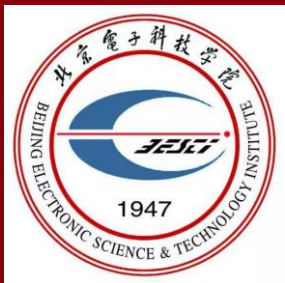
- **一个传输轮次**所经历的时间其实就是**往返时间** RTT
- **传输轮次强调**：把拥塞窗口 $cwnd$ 所允许发送的报文段都连续发送出去，并收到了对已发送的最后一个字节的确认
- 例如：拥塞窗口 $cwnd = 4$ ，这时的往返时间 RTT 就是发送方连续发送 4 个报文段，并收到这 4 个报文段的确认，总共经历的时间



TCP的拥塞控制

慢开始门限 ssthresh

- 防止拥塞窗口 cwnd 增长过大引起网络拥塞
- 用法：
 - 当 $cwnd < ssthresh$ 时，使用慢开始算法
 - 当 $cwnd > ssthresh$ 时，停止使用慢开始算法，改用拥塞避免算法
 - 当 $cwnd = ssthresh$ 时，既可使用慢开始算法，也可使用拥塞避免算法



**谢谢同学们的参与
让我们一起享受计网的饕餮盛宴**